

Dossier rétrospectif

Nous avons travaillé en équipe de 4 afin de réaliser le jeu de plateau Carcassonne.

Le but du projet est de réaliser plusieurs modules permettant d'avoir plusieurs fonctionnalités sur le jeu, à savoir :

- Une logique de jeu.
- Des clients qui peuvent se connecter à un salon de jeu.
- Un enregistreur de partie.
- Un rediffuseur de partie.
- Un affichage.

Au cours de plusieurs semaines de travail, nous avons donc réalisé les tâches suivantes :

- enregistreur
- rediffuser
- client Websocket
- game

Nous allons détailler ci-dessous les différentes implémentations et choix réalisés afin de contribuer au projet.

Logique du jeu

La logique principale du jeu est regroupée dans le package `src/game`. Ce package regroupe les parties principales qui sont le coeur du jeu.

Le plateau

Nous avons décidé de représenter le plateau par une `HashMap` ayant comme clés des positions et comme valeurs des tuiles.

Dans la première conception du plateau, nous avons pensé à un tableau de `72x72` afin d'y placer les tuiles.

Cependant, nous avons vite abandonné l'idée, car cela aurait rendu l'affichage beaucoup trop lourd, ainsi que la logique du plateau en général.

En effet, nous aurions seulement utilisé 72 cases au total sur les plus de 5000 cases disponibles. Cela aurait rendu le jeu très peu optimisé.

Afin de remédier à cela, et pour permettre un affichage plus simple et élégant du plateau, nous avons décidé de définir 4 attributs qui représentent les limites "physiques du plateau". Grâce à ces attributs, nous pouvons directement interagir sur les extrémités du plateau, ce qui permet un affichage réduit et beaucoup plus élégant.

Nous avons donc réalisé deux versions du plateau, mais nous utilisons la v2 pour le projet. La v1 est encore disponible dans `/src/game/plateau/Plateau.java`, mais cette version n'est plus utilisée.

La création du plateau se fait en créant la première tuile du jeu et en la posant directement au centre de celui-ci.

Dans la suite de la partie, on pourra, grâce à l'implémentation décidée, poser les tuiles de façon dynamique sur le plateau mais également les meeples.

Les segments

Pour représenter au mieux nos tuiles, nous avons décidé de mettre en place une logique de segments.

Les segments ont pour rôle de découper nos tuiles en 4 points cardinaux.

Pour chaque point cardinal, on aura un découpage de 3 segments qui vont chacun représenter le type d'élément présent sur celui-ci (une ville, un champ, etc).

On pourra ainsi placer précisément et de manière localisée les meeples directement sur le segment voulu et pas uniquement sur la tuile.

Cela simplifie également la gestion de l'orientation de la tuile, mais surtout sa création. Les segments permettent directement de définir le type de lieu que l'on peut trouver sur la tuile dès la création.

Les différents types de segments sont hérités d'une classe Segment, qui permettent d'ajouter et de modifier facilement la logique côté segment.

Les différents types de segments possibles sont regroupés dans le dossier /SegmentType, chacun des types de segments hérite de la classe mère Segment.

Cela permet de définir facilement de nouveaux types de segments.

Dans la logique de notre jeu, il y a donc 4 types de segments :

- Les segments Abbaye
- Les segments Champ
- Les segments Route
- Les segments Ville

La mise en place de découpage des segments par type est plus naturelle dans la façon de les représenter en jeu.

Orientation et points cardinaux

Au départ, afin de tirer profit des segments, nous avons pensé à une logique qui couplait des points cardinaux et des segments.

Cette classe était nommée Orientation. Elle permettait de pouvoir déplacer beaucoup plus facilement la tuile grâce à ce couplage.

L'orientation était définie selon les 4 points de la tuile, à savoir nord, est, sud et ouest.

Grâce à cela, il était très facile de pivoter la tuile à gauche ou à droite, simplement en décalant les points cardinaux de la tuile dans la direction voulue.

Cette logique de rotation a donc été reprise pour être adaptée à la logique des segments.

Ce ne sont plus les points cardinaux qui tournent mais les segments de la tuile.

Les positions des Tuiles

La position des tuiles est définie de manière assez classique par un x et un y qui permet de représenter la position de la tuile dans le plateau.

Un affichage de la forme (x, y) a été mis en place pour reconnaître la position de la tuile plus facilement.

Direction et points cardinaux

Comme énoncé précédemment, les tuiles étaient initialement pensées pour être définies par des points cardinaux.

Cependant, suite à l'évolution de nos idées, nous avons préféré retenir la logique des segments de tuiles plutôt que les points cardinaux.

Les points cardinaux ont donc été remplacés par une énumération `Direction` qui permet de définir ces points cardinaux.

Les tuiles

Dans nos premières implémentations des tuiles, nous les avions pensées comme composées de 4 points cardinaux qui représentent les bords de la tuile et nous avons donc utilisé une classe `Orientation` composée des 4 points cardinaux et les 4 points cardinaux étaient également composés de segments des types des bords.

Cependant, cette représentation n'était pas optimale comme expliqué précédemment et nous avons donc opté pour une `HashMap` de direction en clés et de `Segments` en valeurs.

Nous avons également pensé les tuiles comme étant définies par une position, mais nous nous sommes orientés vers une implémentation différente. Les tuiles n'ont une position liée que dans le plateau car c'est seulement dans celui-ci que la position d'une tuile nous intéresse et à la création de chaque tuile, aucune d'entre elles n'a de positions.

Trois versions de tuiles ont été implémentées au cours du projet, mais nous nous sommes mis d'accord pour utiliser la V2 des tuiles.

Les autres versions sont toujours disponibles dans `src/game/tuiles` mais ne sont plus utilisées.

Dans la v2 des tuiles, celles-ci sont donc définies par une représentation de leurs bords, des différents segments de celles-ci, et d'un segment `Abbaye` optionnel.

La gestion des meeples n'est plus faite par la tuile, mais par ses segments. Il nous paraît plus logique de placer les meeples sur des bouts de tuiles plutôt que sur la tuile elle-même.

Manager de représentation des tuiles et `TuileFactory`

Afin de faciliter la création des tuiles pour le jeu, nous avons créé une classe

`RepresentationManager`, qui va permettre de lister les différentes représentations des tuiles en respectant la logique du sujet.

Cette représentation est faite par une `HashMap` qui va lier la représentation des tuiles avec le nombre de tuiles à créer de ce type.

Cette `HashMap` sera utilisée par notre `TuileFactory`, qui va alors lire les représentations, le nombre de tuiles à créer, et construire le sachet de tuiles du jeu.

Les joueurs

Dans notre modélisation du jeu, les joueurs possèdent plusieurs attributs dont leur nom, score, le nombre de meeples qu'il leur reste, ainsi qu'une liste de segments.

Cette liste de segments nous permet de parcourir plus rapidement les tuiles qui sont occupées par un joueur.

Un joueur occupe une tuile s'il a un meeples posé sur un des segments de la tuile.

Grâce à cela, on peut parcourir plus facilement les tuiles qui nous intéressent pour la logique du jeu.

On n'est pas obligé de parcourir tout le plateau à chaque fois, mais uniquement la liste de chaque segment détenu par chaque joueur.

Les Meeples

La modélisation du meeples va nous servir à associer différents éléments du jeu.

Le meeples possède un propriétaire, qui va donc être le joueur qui l'a posé.

Il possède également une zone (un segment), qui va représenter le segment sur lequel il est posé.

Nous avons pensé à créer plusieurs types de meeples, afin de représenter leur rôle selon le type de segment dans lequel ils sont posés, afin d'être fidèle à la logique du jeu de base.

Nous avons finalement décidé de ne pas le faire. Le rôle du meeples n'a pas de réel impact dans le déroulement du jeu.

Cela est uniquement utile lors du calcul des scores. Nous avons donc décidé de ne pas surcharger le projet avec des classes qui n'apporteraient pas de réel changement dans le jeu.

Les règles

Pour la partie des règles, nous avons décidé de regrouper toute la logique dans le package `src/game/rules`.

L'objectif était de séparer les règles du jeu du reste de l'implémentation, afin d'avoir une structure plus claire et plus simple à faire évoluer.

Nous avons d'abord mis en place une classe `RegleGenerale`, qui sert de base pour les différentes règles du jeu.

Cette classe permet de définir une méthode commune de validation, qui prend en compte le plateau, une tuile et une position.

L'idée était d'avoir une structure commune pour toutes les règles.

Nous avons ensuite commencé à mettre en place plusieurs types de règles.

Pour le moment, nous avons surtout travaillé sur les règles de placement et les règles de score.

Les règles de placement sont gérées par la classe `ReglePlacement`.

Cette classe a pour rôle de vérifier si une tuile peut être posée sur le plateau à une position donnée.

Elle permet par exemple de vérifier que la position est libre et de préparer la vérification avec les tuiles voisines.

Le but était de faire en sorte que la pose d'une tuile respecte la logique du jeu.

Les règles de score sont gérées à partir de la classe abstraite `RegleScore`.

Cette classe permet de distinguer le score obtenu directement après la pose d'une tuile et le score obtenu en fin de partie.

Cette séparation nous a paru plus logique car dans Carcassonne certaines structures ne rapportent pas de la même façon pendant la partie et à la fin de celle-ci.

Afin de centraliser cette logique, nous avons créé une classe RuleEngine.

Cette classe permet de regrouper les différentes règles de score et de calculer le total des points obtenus.

Cela permet d'avoir une logique plus modulaire et de pouvoir ajouter plus facilement de nouvelles règles par la suite.

Dans l'état actuel du projet, nous avons mis en place deux règles de score principales.

La première est ScoreAbbaye.

Cette règle permet de calculer les points d'une abbaye en comptant le nombre de tuiles présentes autour d'elle.

Si l'abbaye est complètement entourée, elle rapporte 9 points.

Sinon, elle rapporte 1 point plus le nombre de tuiles voisines.

Cette logique est utilisée pendant la partie mais aussi en fin de partie.

La deuxième est ScoreRoute.

Cette règle permet de calculer les points d'une route en parcourant les tuiles reliées entre elles.

Si la route est complète, elle rapporte un nombre de points égal au nombre de tuiles qui composent cette route.

En fin de partie, une route incomplète peut quand même rapporter des points.

Cette règle nous a permis de mettre en place une première logique de parcours du plateau pour suivre une structure connectée.

Nous avons également réalisé des tests sur cette partie afin de vérifier le bon fonctionnement du calcul des scores pour les abbayes, les routes ainsi que du moteur de règles.

Enregistreur

Pour la partie enregistreur, nous avons choisi de mettre en place un script Python capable de se connecter directement au réflecteur afin de conserver l'historique brut d'une partie dans un fichier texte.

L'objectif était d'avoir un composant simple, indépendant du reste de la logique de jeu, et facilement réutilisable pour le rediffuseur.

Nous avons créé une classe Enregistreur qui ouvre une connexion WebSocket vers l'url du réflecteur donnée en paramètre.

Une fois la connexion établie, nous écoutons tous les messages reçus et nous les écrivons immédiatement dans un fichier.

Cela permet de sauvegarder l'ordre exact des événements de la partie, sans transformation du contenu reçu.

Afin de séparer les responsabilités, nous avons isolé la gestion du fichier dans une classe Persistante.

Cette classe s'occupe d'ouvrir le fichier, d'écrire proprement chaque ligne et de fermer la ressource à la fin de l'exécution.

Ce découpage nous a permis d'avoir un enregistreur plus clair, car la connexion réseau et l'écriture dans le fichier sont gérées séparément.

Nous avons également choisi d'écrire les lignes au fur et à mesure de leur réception et de forcer l'écriture sur disque après chaque message.

Ce choix permet d'éviter de perdre l'historique si l'enregistrement s'arrête brutalement avant la fin de la partie.

Enfin, l'enregistreur s'arrête automatiquement lorsqu'il détecte un message de fermeture de partie avec le mot-clé CLOSES.

Nous avons aussi réalisé un petit script de test avec deux clients afin de simuler rapidement plusieurs échanges et vérifier que l'enregistrement du déroulement d'une partie fonctionne correctement.

Rediffuseur

Nous avons décidé de créer une classe rediffuseur qui nous permet d'afficher l'historique d'une partie du jeu grâce à un nom de fichier et à l'url du réflecteur.

Ce rediffuseur affiche l'historique de la partie tant qu'il n'y a plus de ligne à lire dans le fichier et il supprime les espaces inutiles, les lignes vides ainsi que les lignes commentées grâce à la fonction rediffuserFichier de notre classe.

Réflecteur

Le réflecteur utilisé est celui fourni sur le fil.

Nous avons d'abord pensé qu'il fallait le coder nous-mêmes, mais nous nous sommes rendu compte que le réflecteur fourni donnait accès à toutes les fonctionnalités nécessaires pour mener à bien le projet.

Nous avons donc décidé d'utiliser le réflecteur fourni.

Websocket

Le websocket client a connu deux versions différentes.

Version avec serveur et client soit actif, soit passif

Au début de la création du websocket, nous avons distingué 3 fichiers nécessaires afin de pouvoir mettre en place le websocket:

- Le client actif : C'est le client qui aurait pu interagir avec le jeu, une fois son accès autorisé par un admin.

- Le client passif : C'est le client qui aurait été mis en attente lors du déroulement d'une partie déjà active.

- Le serveur : C'est le serveur qui aurait permis de jouer au jeu.

Cette version du websocket a été abandonnée à partir du moment où nous avons compris que le réflecteur agissait lui-même comme un serveur, et que le client passif n'avait pas vraiment d'intérêt et avait un code quasiment identique à celui du client actif. Cette ancienne version est disponible dans /src/util/websocket mais n'est plus utilisée. Nous avons choisi la logique ci-dessous pour la mise en place du websocket.

Version avec client unique, réflecteur et parser de message

C'est la version définitive qui a été retenue par l'équipe et qui se trouve dans src/core. Dans cette logique, nous avons créé un client unique qui va se connecter au réflecteur fourni. Il pourra ainsi écouter les événements et envoyer des commandes. Cette logique est découpée en trois fichiers:

- Le Client : Il va gérer la connexion WebSocket. Il utilise deux threads. Un thread pour l'écoute des messages du réflecteur, et un autre thread pour la saisie des messages de l'utilisateur. Dès sa connexion, le client envoie automatiquement le mot-clé ENTERS. L'utilisateur peut ensuite saisir les commandes qu'il veut, ou alors saisir quit pour se déconnecter.
- La structure de message : Pour faciliter le traitement de l'information par la logique de jeu, chaque texte reçu est converti en un objet Message. L'objet Message découpe la chaîne de caractères en 4 champs : le rang du message (rang), le nom du joueur (source), l'action effectuée (keyword) et la liste des arguments (params).
- Le Parser : C'est l'outil qui va analyser les lignes brutes. Il va découper les messages, ignorer les lignes vides ou les commentaires comme demandé dans le sujet. Il peut être utilisé pour analyser les messages en temps réel du WebSocket, mais aussi pour lire les fichiers de parties lors de l'utilisation du Rediffuseur.

Tâches restant à réaliser

Dans l'état actuel du projet, les tâches qui restent à réaliser pour le projet sont les suivantes:

- Classes liées à la création automatique des tuiles du jeu
- Main pour pouvoir jouer au jeu
- Mise en place d'un arbitre manuel
- Faire en sorte de pouvoir jouer au jeu en réseau
- Interface graphique
- Documentation des classes
- Tests